

An-Najah National University

Faculty of Engineering and Technology

Cybersecurity Department

Networks Programming

PROJECT REPORT

Setting Up Remote Access with Termix on Proxmox VE

Student: **Haneen Baradia**
student ID: **12429915**

Instructor: **Dr. Jehad Hamamreh**

Date: 30th. July. 2026

Abstract

Remote infrastructure management is no longer just about connecting to a single machine, it is about controlling an entire network with confidence and flexibility. This report explores that idea through Termix, a self-hosted remote access platform deployed on Proxmox VE.

What began as a simple experiment, testing SSH, VNC, and Telnet connections to a single Kali Linux machine, quickly grew into something far more ambitious. The project evolved into a fully segmented network, complete with its own router, multiple servers, and real-world features such as secure tunneling, port knocking, Docker container management, and remote wake-up capability.

The result is a hands-on demonstration of how a lightweight, open-source tool can be transformed into the backbone of a realistic, secure, and manageable network environment, bridging the gap between a simple lab exercise and a genuine infrastructure setup.

Table of Contents

Introduction to Proxmox 4
Objectives and Scope 4
Introduction to Termix 5
Preparing the Environment 5
Testing Remote Access Protocols 7
1. **SSH** 7
2. **VNC** 8
3. **Telnet** 9

From a Single Host to a Segmented Network 11
Network Topology 11
Building a Router with OPNsense 12
Placing a Second Host Behind the Router 12
Reaching the Protected Host Through Termix 13
Viewing the Real Screen of a Virtual Machine through Termix 14

Introduction to Proxmox

Proxmox Virtual Environment (Proxmox VE) is an open-source virtualization platform that allows a single physical server to run multiple independent operating systems at the same time. It is classified as a Type-1 hypervisor, which means it is installed directly on the physical hardware rather than running as a program inside another operating system. This gives it direct access to the server's resources, such as CPU, memory, and storage, and allows it to distribute them efficiently among the systems it hosts.

Proxmox supports two main forms of virtualization. The first is full virtualization using KVM, where each virtual machine acts as a completely independent computer with its own operating system, kernel, and hardware emulation. The second is container-based virtualization using LXC, which is lighter in terms of resource usage since containers share the host system's kernel while still keeping their own file systems and processes isolated from each other.

What makes Proxmox particularly useful is its ability to turn a single physical machine into what functions as several real, independent servers. Each virtual machine or container can be assigned its own storage, network configuration, and operating system, and can be started, stopped, backed up, or moved without affecting the others. All of this is managed through a single web-based interface, which handles virtual machines, containers, storage, and networking without requiring any separate software installation.

Objectives and Scope

The primary objective of this project is to design and implement a functional, self-hosted remote infrastructure management environment using Termix, deployed within a virtualized lab built on Proxmox VE. The project is structured in two stages.

The first stage focuses on establishing and validating basic remote access capabilities. This involves deploying Termix and connecting it to a single host, then configuring and testing three distinct remote access protocols: SSH, VNC, and Telnet, to confirm that connections to a remote machine can be reliably established and managed through a single web-based dashboard.

The second stage extends this foundation into a more realistic and complete network environment. Rather than operating a single host on a flat network, this stage introduces network segmentation through a dedicated router, expands the environment to include multiple servers with distinct roles, and incorporates additional real-world functionality, namely secure tunneling and port knocking for controlled external access, Docker container management, and Wake on LAN for remote power control.

Together, these two stages are intended to demonstrate not only the core capabilities of Termix as a remote access tool, but also its practical application within a segmented, multi-host network architecture that reflects common practices in real-world infrastructure management.

Introduction to Termix

Termix is a self-hosted, open-source platform used to manage remote servers from a web browser. It supports several connection types in one place, including SSH, VNC, RDP, and Telnet, so a user does not need a separate application for each protocol.

Besides remote terminal and desktop access, Termix also includes file management, SSH tunneling, and Docker container control. All of these are handled through the same dashboard.

Since Termix is self-hosted, it runs on the user's own machine or server instead of a third-party cloud service. Login details, session logs, and connection data stay local to the environment where Termix is deployed. This is one of the main reasons it was chosen for this project instead of a commercial remote access tool.

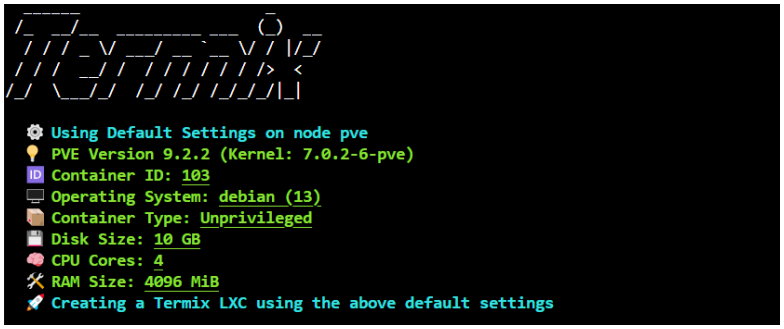
Preparing the Environment

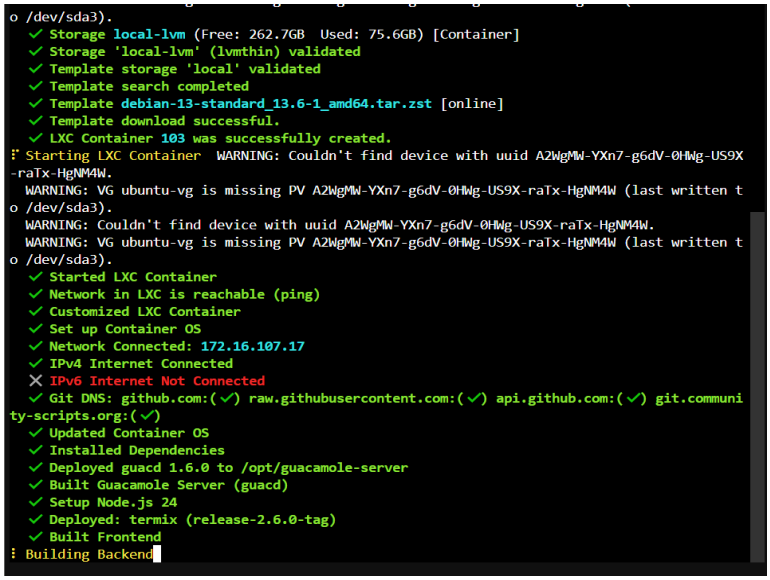
Before testing Termix, two things had to be ready on the Proxmox server: Termix itself, and a machine to connect to and test with.

Setting up Termix:

Proxmox does not include Termix by default, but there is a community project called Proxmox VE Helper-Scripts, which provides ready-made installation scripts for many popular self-hosted applications, including Termix. Instead of creating a container manually and installing everything by hand, the Termix script from this collection was run from the node's Shell.

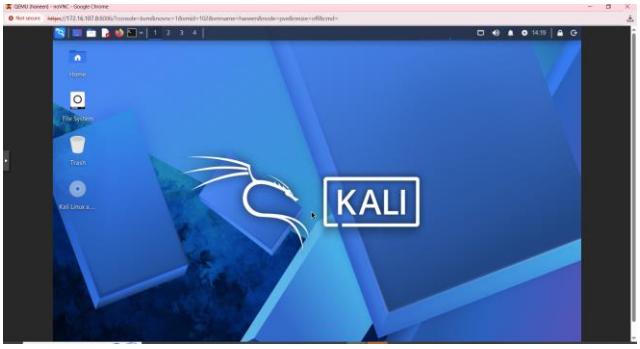
The script created an LXC container based on Debian, unprivileged, with 4 CPU cores, 4GB of RAM, and 10GB of storage. It then installed Node.js and Guacamole (guacd), which Termix uses to support VNC and RDP connections, and built and started both the frontend and backend of the platform. Once the script finished, it printed a local address that opened the Termix login page, where the first administrator account was created.





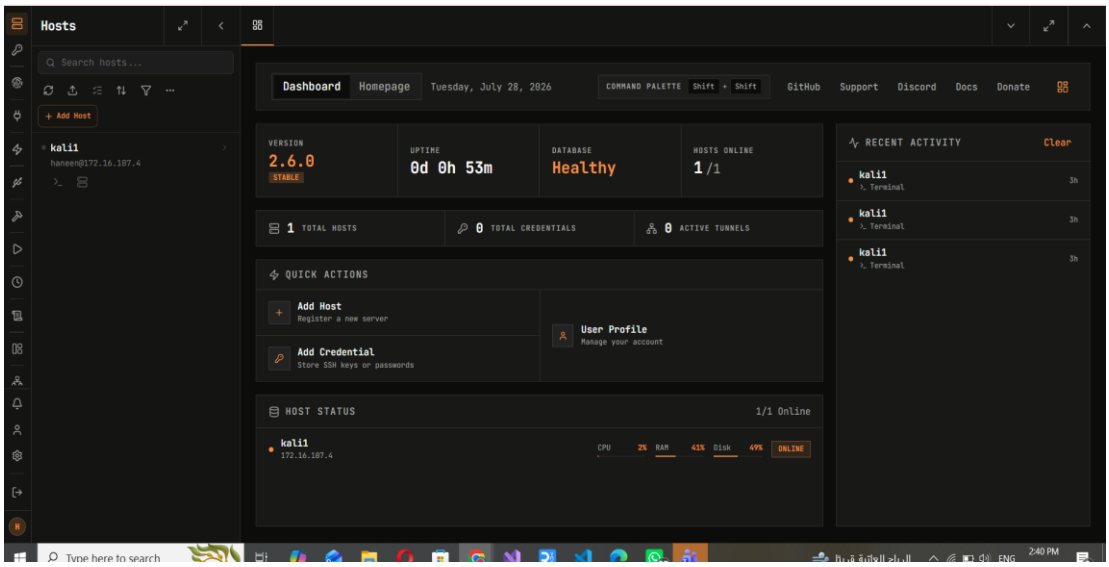
Setting up the target machine

To actually test whether Termix worked, a real machine was needed to connect to, since a working dashboard on its own does not prove anything. For this, a Kali Linux virtual machine was created through Proxmox's normal "Create VM" wizard, with its own disk, CPU, memory, and network interface. This VM was used purely as a target to test the remote access protocols (SSH, VNC, and Telnet) against, to confirm that Termix could



Accessing Termix and Adding the Host

After the installation script finished, it printed a local IP address to open Termix in the browser. The first administrator account was created there, and then the Kali VM was added as a host using its own IP address. The image below shows the Termix dashboard after Kali was added.



Testing Remote Access Protocols

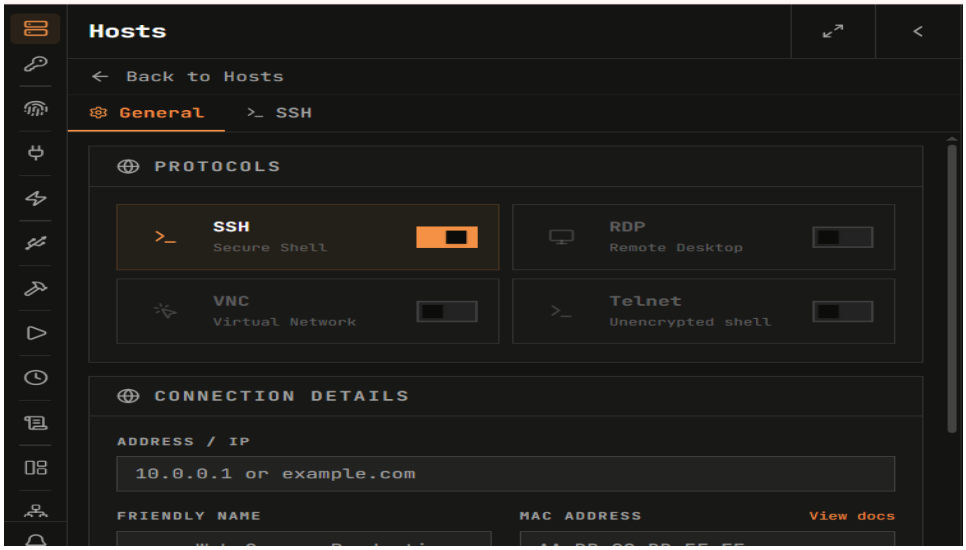
After Termix and the Kali VM were both ready, the next step was to add Kali as a host inside Termix and test it with three protocols: SSH, VNC, and Telnet.

1. SSH

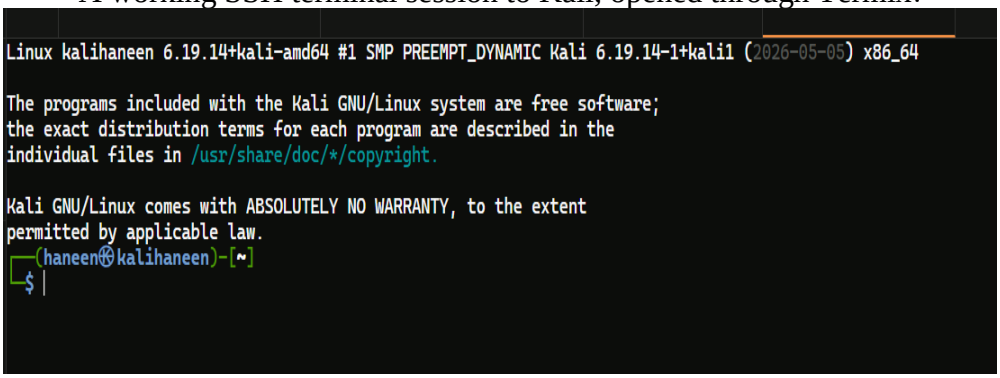
SSH is used to open a secure terminal session on a remote machine. This was the first protocol tested since it is the most basic one.

Kali was added as a host inside Termix, with its IP address and the SSH protocol enabled. The SSH service was installed and started on Kali, and once the connection was set up, it worked correctly, giving a full terminal session inside the browser through Termix.

Installing and enabling the SSH service on Kali:



A working SSH terminal session to Kali, opened through Termix:

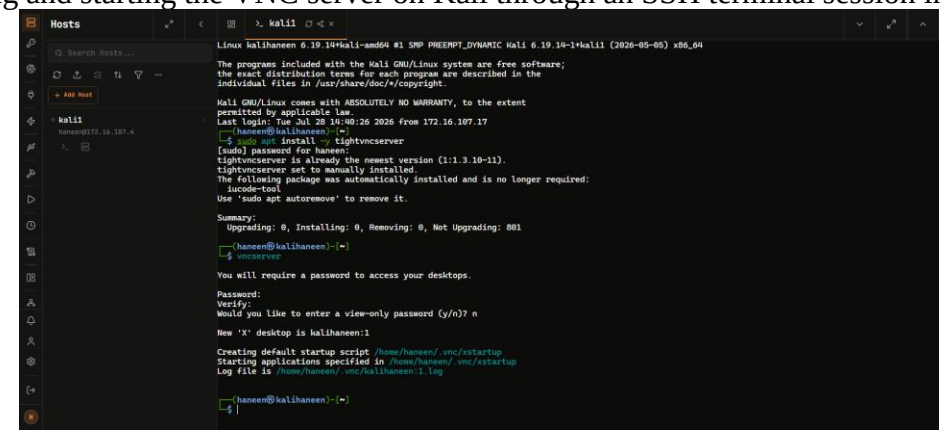


2. VNC

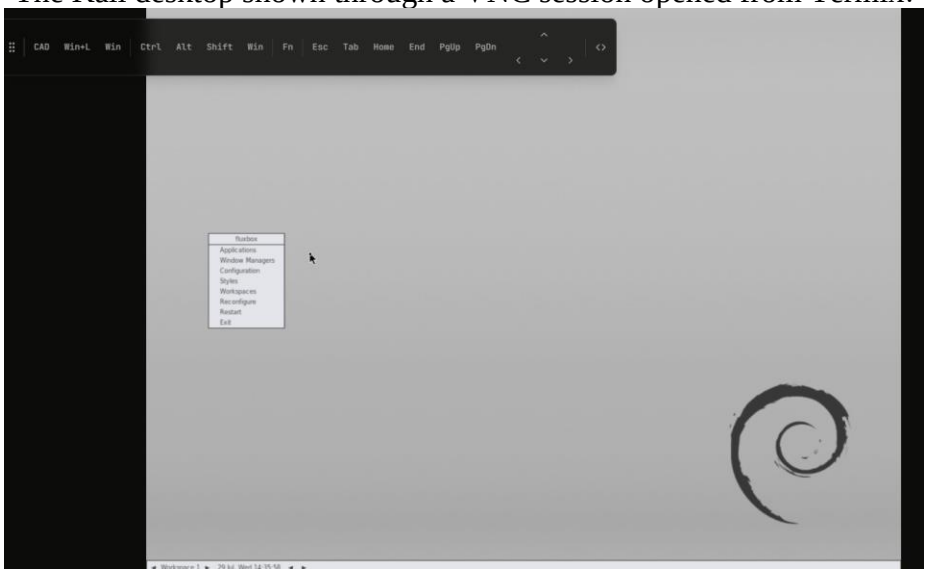
VNC gives full graphical remote desktop access instead of just a text terminal. It was the second protocol tested on Kali.

Tight VNC Server was installed on Kali and configured to run with Fluxbox as the window manager. Kali was then added as a host inside Termix with the VNC protocol enabled, and the connection worked properly, showing the Kali desktop directly inside the browser.

Installing and starting the VNC server on Kali through an SSH terminal session in Termix:



The Kali desktop shown through a VNC session opened from Termix:

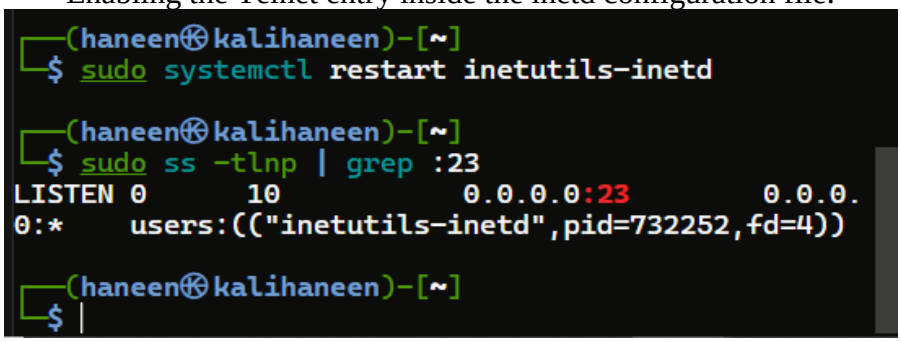


3. Telnet

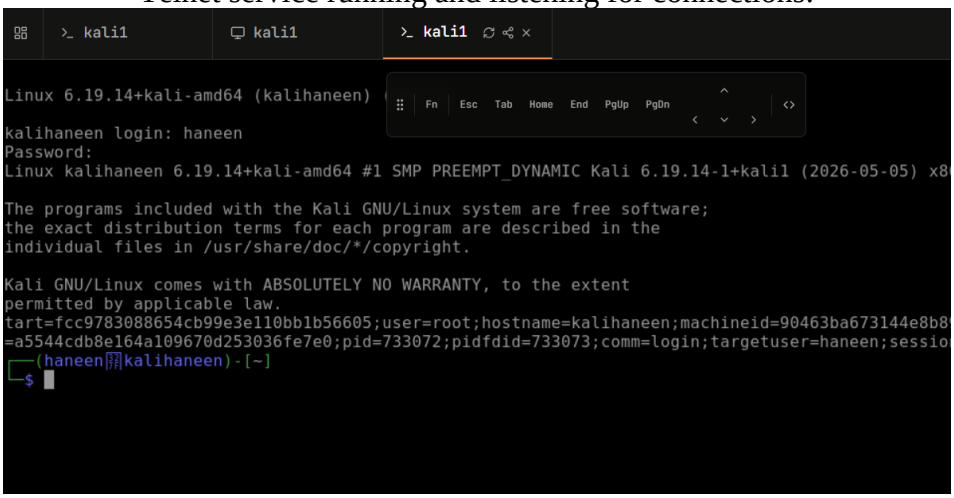
Telnet is an older remote access protocol. Unlike SSH, it sends everything, including passwords, as plain text with no encryption. It was tested here mainly to compare it with SSH.

Setting it up on Kali required inetd, a service that starts programs like Telnet only when a connection comes in. The Telnet entry had to be enabled manually inside its configuration file, since it was disabled by default. After that, Kali was added as a host inside Termix with the Telnet protocol enabled, and the connection worked, reaching Kali's login prompt and then a working command-line session.

Enabling the Telnet entry inside the inetd configuration file:



Telnet service running and listening for connections:



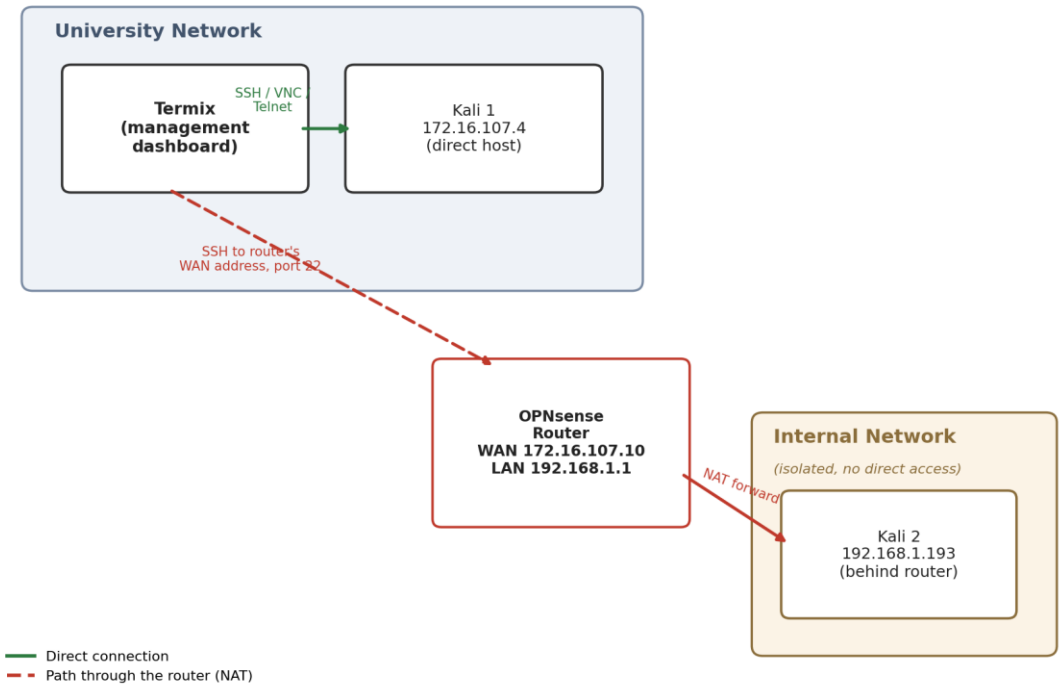
From a Single Host to a Segmented Network

The first part of this project showed that Termix could reliably manage a single host sitting in the open, using SSH, VNC, and Telnet. The next goal was to show that Termix could do the same thing inside a network that actually behaves like a real one: a network where not every host is directly reachable, where a router decides what traffic is allowed in, and where reaching a protected machine requires going through that router on purpose rather than by accident.

Reaching that goal required three things to be built in order: first, a router to create a real boundary between an outside network and an internal one; second, a host placed behind that router instead of beside it, so it would no longer be directly reachable; and third, a controlled path through the router that would let Termix still reach that host on purpose, without removing the protection the router provides. The sections below build these three pieces in that order.

Network Topology

Before building each piece, it helps to see how they fit together. Termix and the first Kali host remain on the university network, exactly as in the first part of this project, and can reach each other directly. The OPNsense router sits between that network and a new, isolated internal network: its WAN interface faces the university network, and its LAN interface faces the internal one. The second Kali host lives only on this internal network, so it has no direct path to the outside. The only way to reach it from Termix is through the router's WAN address, which forwards the connection inward through a NAT rule.



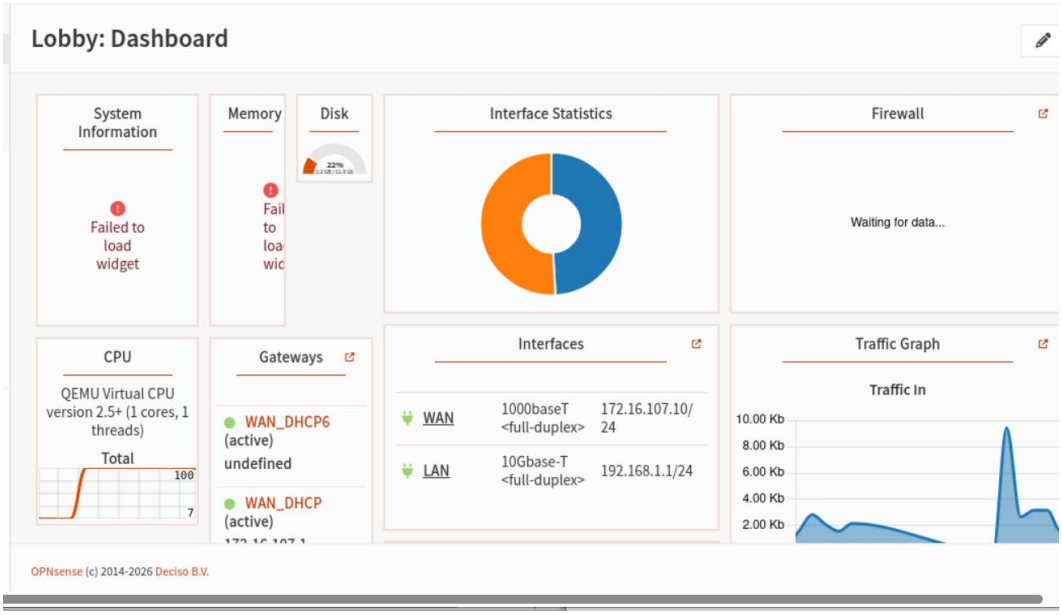
Building a Router with OPNsense

Up to this point, Termix connected to a single host sitting on the same network as the Proxmox server itself, with nothing separating the two. To bring the project closer to how networks are managed in practice, a router was added to create a proper boundary between an internal network and the outside network.

OPNsense was used for this. It is an open-source operating system built specifically to act as a router and firewall, with its own web-based interface for managing traffic. A new virtual machine was created for it with two separate network interfaces: one connected to the same network as the Proxmox server, acting as the WAN side facing the outside, and a second connected to a new, isolated internal network created specifically for this purpose, acting as the LAN side.

Once configured, the router obtained its own address on the outside network while providing a private address range on the internal side, splitting what was previously one flat network into two connected but separate zones.

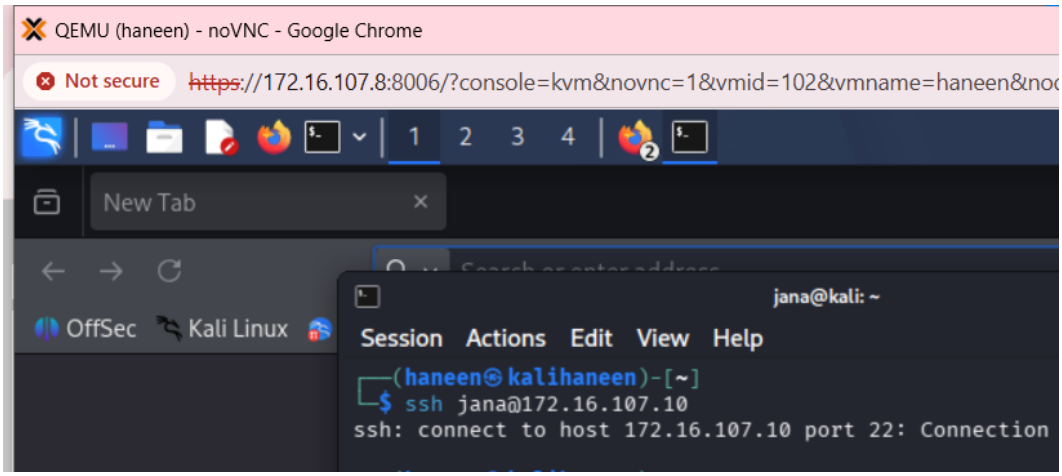
The OPNsense dashboard after configuration, showing the WAN interface connected to the outside network and the LAN interface serving the internal network:



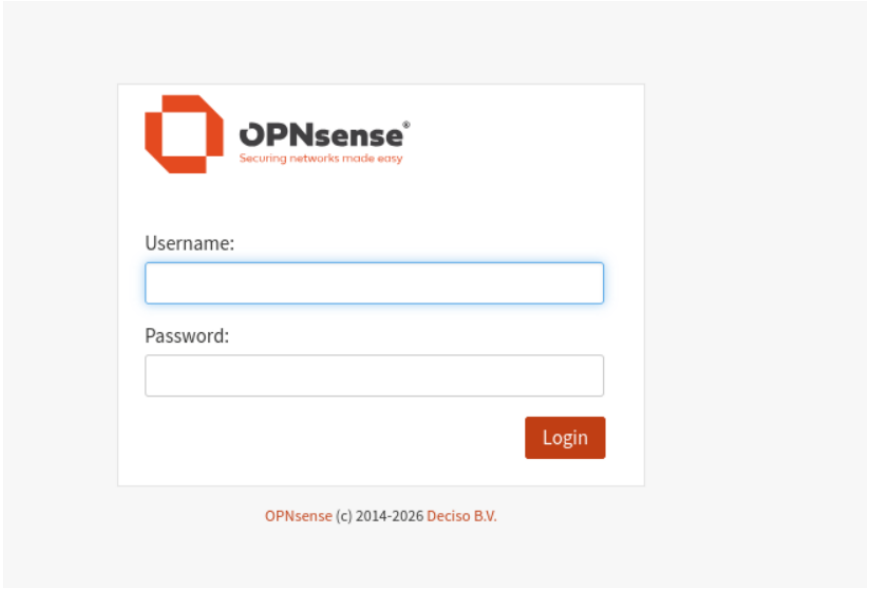
Placing a Second Host Behind the Router

With the router in place, a second Kali Linux virtual machine was connected to the new internal network instead of the outside network. This host received its network settings automatically from the router rather than from the university network, placing it fully behind the router rather than beside it.

The second host receiving an internal address from the router:



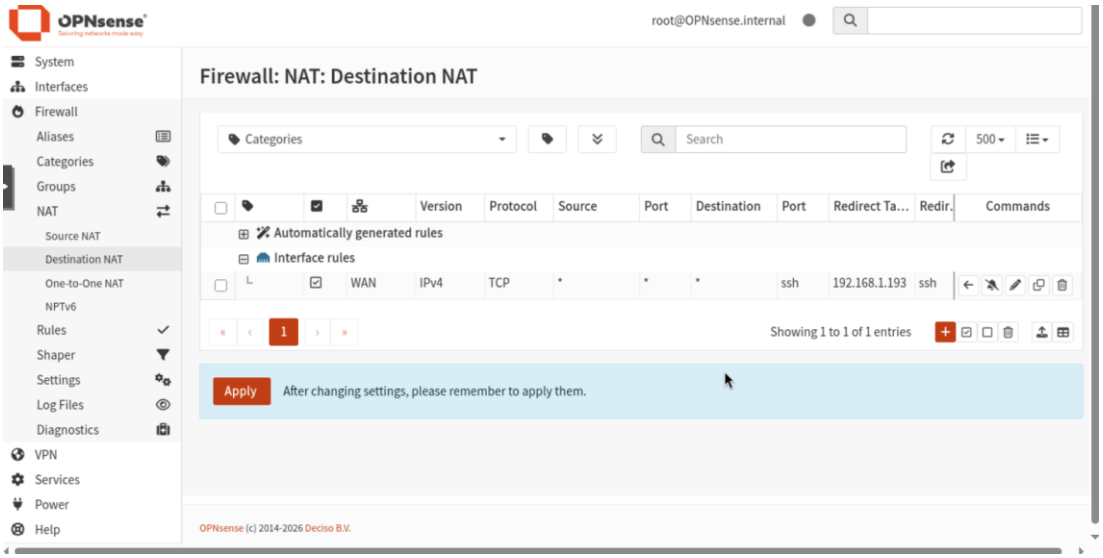
Because this host now sits behind the router, it is not directly reachable from the outside network. It can, however, reach the router itself, which manages all traffic in and out of the internal network on its behalf. This was confirmed by opening the router's web login page directly from the second host:



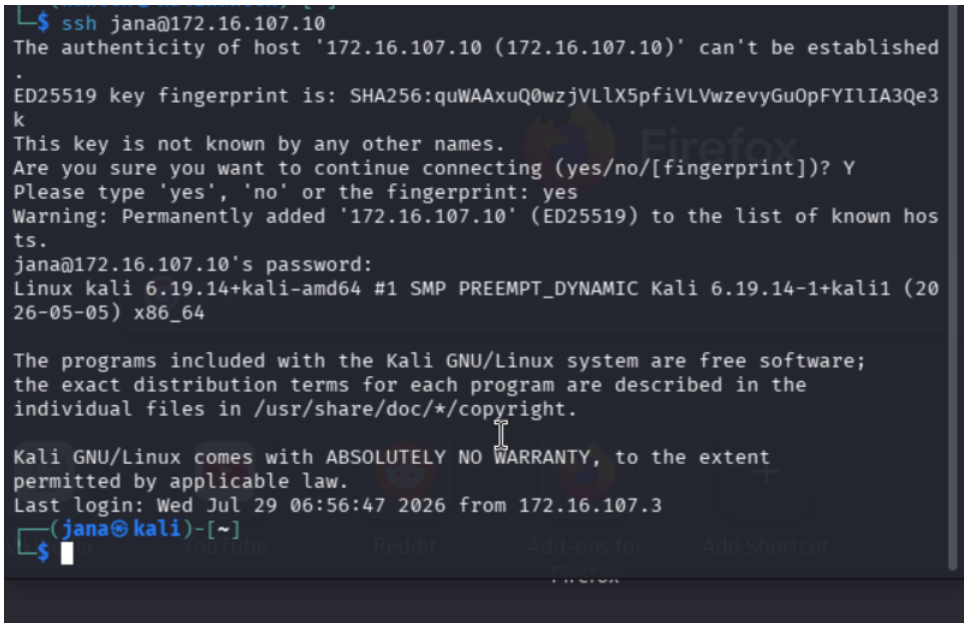
Reaching the Protected Host Through Termix

Placing the second host behind the router means Termix, which sits outside on the university network, can no longer reach it directly the way it reaches the first host. To allow controlled access from Termix without exposing the internal network directly, a NAT rule was configured on the router. This rule forwards SSH traffic arriving at the router's own outside address to the second host's internal address, so the router acts as a controlled gateway rather than removing the separation entirely.

The NAT rule forwarding SSH traffic through the router to the second host:



With this rule in place, the second host was added to Termix using the router's own address instead of its internal address. The connection succeeded, giving a full terminal session to the second host through the router, exactly as with the first host, but this time reaching a machine that is otherwise isolated from the outside network:

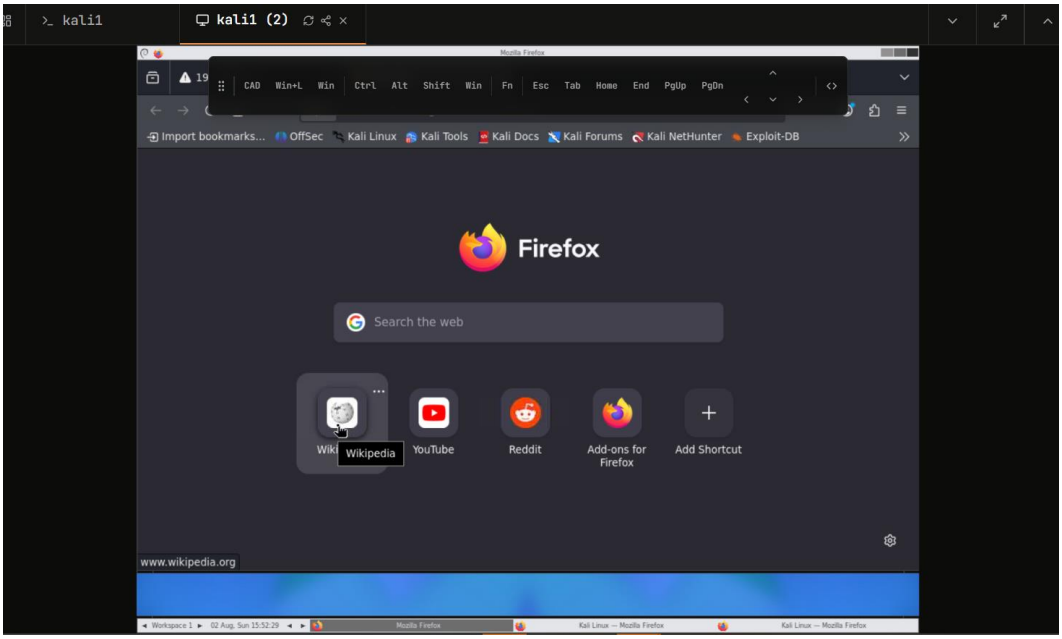


Viewing the Real Screen of a Virtual Machine through Termix

For the first Kali VM, the graphical session opened by Termix was a separate virtual display created specifically for VNC, independent from the display shown on the virtual machine's own console in Proxmox. To confirm that Termix could control the real screen of a machine rather

than a separate one, a full desktop environment was installed and set to run on the machine's actual display, and a VNC server was configured to expose that exact display instead of creating a new one.

A browser opened through Termix, running directly on the virtual machine's real screen:



The same session shown at the same time through Termix and through the virtual machine's own console in Proxmox, confirming both display the same live desktop rather than two separate ones:

